

Laporan Tugas Kecil 3

IF2211 STRATEGI ALGORITMA

SEMESTER II TAHUN 2025/2026



Ice Sliding Puzzle Solver dengan Algoritma Pathfinding

Disusun Oleh:

13524085 Ariel Cornelius Sitorus

13524090 Nashiruddin Akram

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Mei 2026

Contents

1	Pendahuluan	2
2	Pemodelan Permasalahan	2
2.1	State dan Goal	2
2.2	Pembangkitan Successor	2
3	Algoritma Pathfinding	2
3.1	Uniform Cost Search	2
3.2	Greedy Best First Search	3
3.3	A* Search	3
3.4	Bonus: BFS	3
3.5	Bonus: IDA*	3
4	Heuristik	3
4.1	H1: Manhattan Distance	3
4.2	H2: Minimum Slide	4
4.3	H3: Dijkstra ke Target Berikutnya	4
4.4	Admissibility	4
5	Perbandingan Teoritis	4
6	Source Program dan Cara Menjalankan	5
7	Tangkapan Layar	5
8	Hasil Percobaan	7
9	Analisis Hasil Percobaan	8
10	Pranala Repository	9
11	Kesimpulan	9
A	Checklist Spesifikasi	10
B	Acuan	10

1 Pendahuluan

Ice Sliding Puzzle adalah permainan pencarian rute pada papan es. Aktor berawal dari tile Z dan harus mencapai tile tujuan 0. Pada setiap aksi, aktor memilih arah U, D, L, atau R, lalu terus meluncur sampai tepat sebelum menabrak dinding X. Gerakan tidak valid apabila aktor keluar papan, melewati lava L, melewati checkpoint angka yang belum sesuai urutan, atau berhenti di goal sebelum semua checkpoint selesai.

Setiap tile memiliki cost traversal. Cost satu gerakan adalah jumlah cost semua tile yang dilalui selama sliding, tidak termasuk tile posisi awal sebelum gerakan. Karena itu, solusi dengan jumlah gerakan paling sedikit belum tentu merupakan solusi dengan total cost terkecil.

Program dibuat dengan backend Go dan frontend HTML/CSS/JavaScript. Backend membaca testcase, memvalidasi papan, menjalankan algoritma pathfinding, dan mengirimkan hasil serta frame visualisasi ke GUI.

2 Pemodelan Permasalahan

2.1 State dan Goal

Satu state pencarian direpresentasikan sebagai:

$$n = (\text{posisi aktor, checkpoint berikutnya})$$

Posisi aktor menyimpan baris dan kolom saat ini. Checkpoint berikutnya menyimpan indeks checkpoint terkecil yang belum dilewati. State awal adalah posisi Z dengan checkpoint berikutnya bernilai 0. State goal tercapai jika aktor tepat berhenti di 0 dan seluruh checkpoint sudah dilewati sesuai urutan.

2.2 Pembangkitan Successor

Dari sebuah state, program mencoba empat arah gerak. Mekanisme sliding berjalan tile demi tile sampai aktor menabrak dinding. Jika gerakan valid, posisi tepat sebelum dinding menjadi state baru. Jika aktor keluar papan, melewati lava, atau melewati checkpoint yang belum sesuai urutan, successor dibatalkan.

Cost successor dihitung dari akumulasi cost tile yang dilalui selama gerakan. Urutan ekspansi arah yang digunakan program adalah U, D, L, lalu R.

3 Algoritma Pathfinding

Notasi yang digunakan mengikuti materi Route Planning. $g(n)$ adalah cost dari start ke state n , $h(n)$ adalah estimasi cost dari state n ke target, dan $f(n)$ adalah fungsi evaluasi yang menentukan prioritas ekspansi.

3.1 Uniform Cost Search

Uniform Cost Search (UCS) memakai priority queue berdasarkan cost aktual:

$$f(n) = g(n)$$

Langkahnya adalah memasukkan state awal dengan $g(n) = 0$, mengambil node dengan $g(n)$ terkecil, mengecek apakah node tersebut goal, lalu membangkitkan successor dari empat arah. Untuk setiap successor valid, program menghitung $g'(n) = g(n) + \text{cost}$ gerakan dan memasukkannya ke priority queue. Karena cost tidak negatif dan ruang state berhingga, UCS lengkap dan optimal terhadap total cost.

3.2 Greedy Best First Search

Greedy Best First Search (GBFS) memakai prioritas:

$$f(n) = h(n)$$

GBFS memilih state yang terlihat paling dekat ke target menurut heuristik. Program tetap menyimpan $g(n)$ untuk menghitung cost akhir, tetapi nilai tersebut tidak memengaruhi prioritas ekspansi. GBFS sering meninjau state lebih sedikit, tetapi tidak menjamin solusi optimal karena cost yang sudah ditempuh diabaikan.

3.3 A* Search

A* memakai gabungan cost aktual dan estimasi:

$$f(n) = g(n) + h(n)$$

Setiap successor diberi prioritas berdasarkan total cost yang sudah ditempuh ditambah estimasi cost tersisa. Jika heuristik admissible, A* dapat menemukan solusi optimal. Dalam program ini, A* dapat memakai H1 Manhattan, H2 Minimum Slide, atau H3 Dijkstra.

3.4 Bonus: BFS

Breadth First Search (BFS) memakai queue FIFO dan mengekspansi state berdasarkan kedalaman. Pada puzzle ini, BFS mencari solusi dengan jumlah gerakan sliding paling sedikit. BFS tidak optimal terhadap total cost ketika tiap tile memiliki cost berbeda.

3.5 Bonus: IDA*

Iterative Deepening A* (IDA*) memakai $f(n) = g(n) + h(n)$, tetapi menjalankan DFS dengan threshold. Threshold awal adalah $h(\text{start})$. Jika goal belum ditemukan, threshold dinaikkan ke nilai f terkecil yang sebelumnya melewati batas. IDA* hemat memori, tetapi dapat mengekspansi ulang state pada iterasi threshold berikutnya.

4 Heuristik

4.1 H1: Manhattan Distance

H1 menjumlahkan Manhattan distance dari posisi aktor ke checkpoint berikutnya, antar checkpoint yang tersisa, lalu dari checkpoint terakhir ke goal. Jumlah jarak tersebut dikalikan dengan cost walkable minimum:

$$h_1(n) = \left(\sum \text{jarak Manhattan target tersisa} \right) \times \text{minCost}$$

H1 murah dihitung, tetapi sering terlalu optimistis karena tidak memperhitungkan rintangan dan aturan sliding.

4.2 H2: Minimum Slide

H2 menghitung jumlah slide minimum untuk mencapai target berikutnya melalui BFS ringan pada state puzzle, lalu mengalikannya dengan cost walkable minimum:

$$h_2(n) = \text{minSlideToNextTarget}(n) \times \text{minCost}$$

H2 lebih sadar terhadap aturan sliding dibanding H1, tetapi tetap lebih ringan daripada H3.

4.3 H3: Dijkstra ke Target Berikutnya

H3 menjalankan Dijkstra dari state saat ini menuju target berikutnya dengan aturan sliding dan cost asli. Jika masih ada checkpoint, targetnya adalah checkpoint berikutnya; jika checkpoint sudah habis, targetnya adalah goal. H3 lebih informatif, tetapi lebih mahal karena melakukan pencarian internal pada setiap evaluasi heuristik.

4.4 Admissibility

Ketiga heuristik yang digunakan pada A* bersifat admissible dengan asumsi cost tile walkable tidak negatif. H1 adalah lower bound karena setiap rute valid harus menempuh minimal jarak Manhattan antar target dan setiap tile memiliki cost minimal minCost. H2 adalah lower bound karena setiap slide valid melewati minimal satu tile dengan cost minimal minCost. H3 adalah lower bound karena hanya menghitung cost minimum menuju target berikutnya, sedangkan solusi penuh dari state tersebut harus setidaknya mencapai target berikutnya sebelum selesai.

Dengan demikian, heuristik tidak melebihi-lebihkan cost sisa menuju goal. H3 biasanya paling kuat, H1 paling murah, dan H2 berada di tengah.

5 Perbandingan Teoritis

Algoritma	Prioritas	Optimal Cost	Memori	Catatan
UCS	$g(n)$	Ya	Besar	Acuan solusi optimal untuk cost non-negatif.
BFS	Kedalaman	Tidak	Sedang-besar	Optimal jumlah move, bukan total cost.
GBFS	$h(n)$	Tidak	Sedang	Biasanya cepat, tetapi dapat memilih rute mahal.
A*	$g(n) + h(n)$	Ya, jika heuristik admissible	Besar	Menyeimbangkan UCS dan arahan heuristik.
IDA*	Threshold $g(n) + h(n)$	Ya dengan heuristik admissible	Kecil	Hemat memori, tetapi dapat mengulang ekspansi.

Table 1: Perbandingan teoritis algoritma yang diimplementasikan.

6 Source Program dan Cara Menjalankan

Source program terdapat pada folder `src`. Struktur utama program:

<code>src/backend/algorithm</code>	UCS, GBFS, A*, BFS, IDA*, dan heuristik
<code>src/backend/engine</code>	Parser, board, state, dan mekanisme sliding
<code>src/backend/main.go</code>	HTTP server dan endpoint API
<code>src/frontend</code>	GUI visualizer
<code>test</code>	Testcase <code>.txt</code>
<code>dox</code>	Laporan LaTeX

Program dijalankan dengan:

```
.\run.ps1
```

Script tersebut menghentikan proses lama pada port 8080, lalu menjalankan:

```
go run ./src/backend
```

Setelah server hidup, GUI dibuka melalui `http://localhost:8080`. Pengujian laporan ini memakai `go1.25.4 windows/amd64`.

7 Tangkapan Layar

Bagian ini berisi placeholder sesuai permintaan. Setelah screenshot final tersedia, placeholder dapat diganti dengan gambar asli.

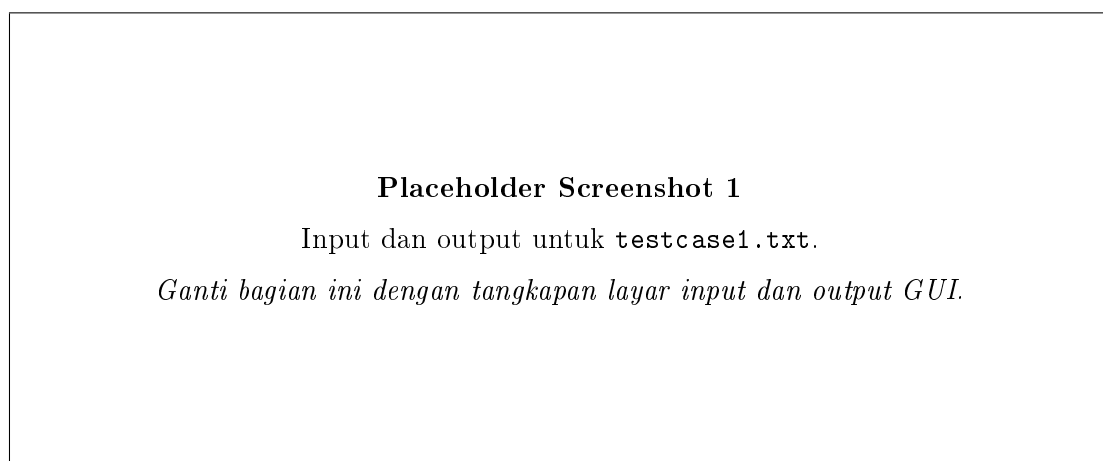


Figure 1: Input dan output untuk `testcase1.txt`.

Placeholder Screenshot 2

Input dan output untuk `testcase5.txt`.

Ganti bagian ini dengan tangkapan layar input dan output GUI.

Figure 2: Input dan output untuk `testcase5.txt`.

Placeholder Screenshot 3

Input dan output untuk `testcase6.txt`.

Ganti bagian ini dengan tangkapan layar input dan output GUI.

Figure 3: Input dan output untuk `testcase6.txt`.

Placeholder Screenshot 4

Input dan output untuk `testcase8.txt`.

Ganti bagian ini dengan tangkapan layar input dan output GUI.

Figure 4: Input dan output untuk `testcase8.txt`.

Placeholder Screenshot 5

Input dan output untuk `testcase9.txt`.

Ganti bagian ini dengan tangkapan layar input dan output GUI.

Figure 5: Input dan output untuk `testcase9.txt`.

Placeholder Screenshot 6

Kasus tidak ada solusi pada `testcase10.txt`.

Ganti bagian ini dengan tangkapan layar input dan output GUI.

Figure 6: Kasus tidak ada solusi pada `testcase10.txt`.

8 Hasil Percobaan

Percobaan dilakukan melalui backend solver yang sama dengan GUI. Waktu eksekusi hanya mengukur proses pencarian, tidak termasuk input pengguna atau rendering GUI.

Table 2: Hasil percobaan algoritma utama dan bonus BFS dengan H2 untuk algoritma informed.

Testcase	Algoritma	Found	Cost	Move	Iterasi	Waktu (ms)
testcase1.txt	UCS	Ya	87	8	11	0
testcase1.txt	BFS	Ya	87	8	13	0
testcase1.txt	GBFS-H2	Ya	87	8	10	0
testcase1.txt	A*-H2	Ya	87	8	11	0
testcase5.txt	UCS	Ya	191	7	22	0
testcase5.txt	BFS	Ya	191	7	22	0
testcase5.txt	GBFS-H2	Ya	200	8	13	0
testcase5.txt	A*-H2	Ya	191	7	22	0

Testcase	Algoritma	Found	Cost	Move	Iterasi	Waktu (ms)
testcase6.txt	UCS	Ya	217	14	93	0
testcase6.txt	BFS	Ya	218	13	89	0
testcase6.txt	GBFS-H2	Ya	233	14	25	0
testcase6.txt	A*-H2	Ya	217	14	54	0
testcase7.txt	UCS	Ya	277	12	66	0
testcase7.txt	BFS	Ya	277	12	72	0
testcase7.txt	GBFS-H2	Ya	277	12	25	0
testcase7.txt	A*-H2	Ya	277	12	41	0
testcase8.txt	UCS	Ya	1084	28	262	1
testcase8.txt	BFS	Ya	1084	28	181	0
testcase8.txt	GBFS-H2	Ya	1084	28	158	4
testcase8.txt	A*-H2	Ya	1084	28	258	8
testcase9.txt	UCS	Ya	1313	39	311	0
testcase9.txt	BFS	Ya	1385	29	302	1
testcase9.txt	GBFS-H2	Ya	1385	29	74	3
testcase9.txt	A*-H2	Ya	1313	39	240	13
testcase10.txt	UCS	Tidak	0	0	14	0
testcase10.txt	BFS	Tidak	0	0	14	0
testcase10.txt	GBFS-H2	Tidak	0	0	14	0
testcase10.txt	A*-H2	Tidak	0	0	14	0

Testcase	Algoritma	Cost	Move	Iterasi	Waktu (ms)
testcase6.txt	GBFS-H1	217	14	15	0
testcase6.txt	GBFS-H2	233	14	25	3
testcase6.txt	GBFS-H3	217	14	30	1
testcase6.txt	A*-H1	217	14	90	1
testcase6.txt	A*-H2	217	14	54	1
testcase6.txt	A*-H3	217	14	30	1
testcase9.txt	GBFS-H1	2046	47	66	0
testcase9.txt	GBFS-H2	1385	29	74	11
testcase9.txt	GBFS-H3	1408	42	73	26
testcase9.txt	A*-H1	1313	39	306	3
testcase9.txt	A*-H2	1313	39	240	40
testcase9.txt	A*-H3	1313	39	149	25

Table 3: Perbandingan heuristik H1, H2, dan H3 pada GBFS dan A*.

9 Analisis Hasil Percobaan

Pada testcase kecil seperti `testcase1.txt`, semua algoritma menemukan solusi dengan cost dan jumlah move yang sama. Ruang state yang kecil membuat perbedaan iterasi belum terlalu terlihat.

Pada `testcase6.txt`, BFS menemukan solusi 13 move dengan cost 218, sedangkan UCS dan A* menemukan cost 217 dengan 14 move. Ini menunjukkan bahwa solusi dengan move paling sedikit tidak selalu memiliki total cost paling kecil.

GBFS sering lebih hemat iterasi. Pada `testcase9.txt`, GBFS-H2 hanya meninjau 74 iterasi, sedangkan UCS meninjau 311 iterasi. Namun, cost GBFS-H2 adalah 1385, lebih besar dari UCS dan A*-H2 yang menghasilkan cost 1313. GBFS-H1 bahkan menghasilkan cost 2046. Hal ini sesuai teori bahwa GBFS cepat tetapi tidak optimal.

A* menghasilkan cost yang sama dengan UCS pada seluruh testcase solvable yang diuji. Pada `testcase9.txt`, A*-H2 mengurangi iterasi dari 311 menjadi 240, dan A*-H3 menurunkannya lagi menjadi 149. Walaupun begitu, H3 tidak selalu menjadi waktu tercepat karena perhitungan heuristiknya menjalankan Dijkstra internal.

`testcase10.txt` adalah kasus tanpa solusi. Semua algoritma yang diuji mengembalikan Found = Tidak. Iterasi yang sama terjadi karena goal terisolasi dan ruang state reachable kecil.

10 Pranala Repository

Repository kode program:

https://github.com/Sakazu01/Tucil3_13524085_13524090

11 Kesimpulan

Program berhasil memodelkan Ice Sliding Puzzle sebagai pencarian state dengan aturan sliding, checkpoint berurutan, lava, dan cost traversal. UCS menjadi pembanding optimal berbasis cost. GBFS cenderung cepat tetapi tidak optimal. A* dengan heuristik admissible mempertahankan optimalitas UCS sambil mengurangi ekspansi pada beberapa testcase. Bonus yang dikerjakan mencakup GUI, algoritma tambahan BFS dan IDA*, serta dua heuristik tambahan selain heuristik utama.

A Checklist Spesifikasi

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan.	✓	
2	Program berhasil dijalankan.	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan.	✓	
4	Program dapat membaca masukan berkas <code>.txt</code> serta menyimpan solusi dalam berkas <code>.txt</code> .	✓	
5	Program memiliki Graphical User Interface (GUI).	✓	
6	Ada algoritma pathfinding alternatif selain spesifikasi wajib, yaitu BFS dan IDA*.	✓	
7	Ada dua alternatif heuristik selain heuristik utama, yaitu H2 Minimum Slide dan H3 Dijkstra.	✓	

Table 4: Checklist pemenuhan spesifikasi.

B Acuan

- [1] Spesifikasi Tugas Kecil 3 IF2211 Strategi Algoritma 2025/2026, Ice Sliding Puzzle Solver.
- [2] Materi kuliah IF2211, Route Planning Bagian 1: BFS, DFS, UCS, Greedy Best First Search.
- [3] Materi kuliah IF2211, Route Planning Bagian 2: Algoritma A*.